

به نام پروردگار هدایت کننده به راه راست



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

پروژه درس آزمون نرم افزار

گزارش ارزیابی مدل های زبانی بزرگ

در تولید تست کیس ها با استفاده از بنچمارک TestEval

استاد : جناب آقای دکتر محمدرضا شعرباف

محدثه آخوندی

4003613002

خرداد ماه 1404

۱. مقدمه

با پیشرفت فناوری‌های مبتنی بر هوش مصنوعی، مدل‌های زبانی بزرگ (LLMs) به ابزارهایی قدرتمند در تولید، اصلاح و تحلیل کدهای برنامه‌نویسی تبدیل شده‌اند. یکی از مهم‌ترین کاربردهای این مدل‌ها، استفاده در فرآیند تولید تست‌کیس برای بررسی صحت عملکرد برنامه‌هاست. تولید تست‌کیس به معنای ایجاد ورودی‌هایی است که بتوانند بخش‌های مختلف کد را اجرا کرده و رفتار آن را در شرایط متنوع بررسی کنند. این فرآیند در توسعه نرم‌افزار بسیار حیاتی است اما معمولاً زمان‌بر و پرهزینه تلقی می‌شود.

در این راستا، مقاله حاضر به بررسی توانایی مدل‌های زبانی در تولید خودکار تست‌کیس‌ها می‌پردازد. نویسندگان مقاله با معرفی یک بنچمارک جدید به نام TESTEVAL، تلاش کرده‌اند چارچوبی منسجم و دقیق برای سنجش کیفیت تست‌کیس‌های تولیدشده توسط LLM‌ها طراحی کنند. این مقاله نه تنها به ارزیابی عملکرد مدل‌ها در تولید تست‌های کلی می‌پردازد، بلکه توانایی آن‌ها را در تولید تست‌های هدفمند و پوشش مسیرهای خاص اجرایی بررسی می‌کند.

۲. هدف پژوهش

هدف اصلی پژوهش طراحی و توسعه یک بنچمارک جامع برای ارزیابی عملکرد مدل‌های زبانی در زمینه تولید تست‌کیس است. در واقع، نویسندگان به دنبال پاسخ به این سؤال هستند که آیا مدل‌های زبانی قادرند ورودی‌هایی تولید کنند که خطوط، شاخه‌ها و مسیرهای خاصی از یک برنامه را به صورت دقیق پوشش دهند یا خیر. آن‌ها همچنین می‌خواهند بدانند که این مدل‌ها تا چه اندازه می‌توانند تست‌کیس‌هایی متنوع، دقیق و اجرایی تولید کنند و آیا می‌توان از آن‌ها در فرآیند تست واقعی نرم‌افزار بهره گرفت.

۳. طراحی بنچمارک TESTEVAL

بنچمارک TESTEVAL بر اساس ۲۱۰ برنامه پایتون از پلتفرم LeetCode طراحی شده است. این برنامه‌ها بر اساس شاخصی به نام پیچیدگی سیکلوماتیک انتخاب شده‌اند تا ساختارهای منطقی متنوع و چالش‌برانگیزی داشته باشند. این معیار با استفاده از گراف جریان کنترل برنامه محاسبه می‌شود و نشان‌دهنده تعداد شاخه‌ها و مسیرهای موجود در کد است. هدف از این انتخاب آن بوده که برنامه‌هایی انتخاب شوند که واقعاً نیازمند تست‌کیس‌های متنوع باشند و نه صرفاً چند خط ساده.

Cyclomatic Complexity :

$$V(G) = E - N + P$$

E = تعداد یال‌ها (edges)

N = تعداد گره‌ها (nodes)

P = تعداد اجزای هم‌بند (connected components)

این معیار نشان می‌دهد که برنامه چه تعداد مسیر اجرایی مختلف دارد. هرچه بالاتر، تست‌نویسی سخت‌تر.

هر برنامه شامل توضیح متنی، کد منبع و مجموعه‌ای از شاخه‌ها و مسیرهای مشخص برای ارزیابی است. برای هر برنامه، مراحل پیش‌پردازشی مانند حذف کامنت‌ها، ادغام خطوط شکسته، اضافه کردن import های لازم و تبدیل عبارات شرطی درون خطی به بلوک‌های چندخطی انجام شده تا کد آماده اجرای دقیق و تحلیل پوشش باشد.

۴. وظایف تعریف شده در TESTEVAL

در چارچوب TESTEVAL سه وظیفه مستقل طراحی شده است. اولین وظیفه، تولید تست کیس برای پوشش کلی کد است که به آن Overall Coverage گفته می‌شود. در این حالت، مدل باید تست‌هایی تولید کند که هرچه بیشتر خطوط و شاخه‌های موجود در برنامه را پوشش دهند. وظیفه دوم، تولید تست برای پوشش خطوط یا شاخه‌های خاص (Targeted Line/Branch Coverage) است. در این وظیفه، اطلاعات هدف مانند شماره خط یا شاخه مشخص به مدل داده می‌شود و از آن خواسته می‌شود تستی بنویسد که دقیقاً آن بخش از کد را اجرا کند. وظیفه سوم، تولید تست برای مسیرهای خاص اجرایی (Targeted Path Coverage) است. این وظیفه پیچیده‌ترین حالت است، زیرا مدل باید دنباله‌ای از شرایط منطقی را طی کند تا به مسیر خاصی برسد که ممکن است شامل چندین شاخه پی‌درپی باشد.

۵. روش ارزیابی عملکرد مدل‌ها

پس از تولید تست کیس‌ها، آن‌ها از سه جنبه مورد بررسی قرار می‌گیرند. ابتدا بررسی نحوی (Syntax) انجام می‌شود تا مشخص شود کد تولیدشده از نظر نگارشی صحیح است یا خیر. سپس بررسی اجرایی (Execution) انجام می‌شود تا تست بدون خطا اجرا شود. در نهایت، بررسی درستی عبارات Assert در تست کیس (Assertion Correctness) انجام می‌شود. البته اگر Assertion نادرست باشد ولی تست اجرا شده و مسیر مدنظر را طی کند، آن تست همچنان برای تحلیل پوشش قابل قبول است.

برای سنجش میزان پوشش، از ابزار pytest-cov استفاده شده است. علاوه بر این، شاخص جدیدی به نام cov@k نیز معرفی شده است. این معیار نشان می‌دهد که اگر فقط k تست کیس از میان تست‌های تولیدشده را انتخاب کنیم، چقدر از کد پوشش داده می‌شود. این معیار برای سنجش تنوع تست‌های تولیدشده توسط مدل بسیار مفید است.

محاسبه cov@k برای یک برنامه طبق مقاله :

M تست قابل اجرا داریم.

اون‌ها رو به $[M/k]$ گروه k-تایی تقسیم می‌کنیم.

برای هر گروه، coverage می‌گیریم.

cov@k = میانگین coverage همه‌ی گروه‌ها.

۶. نتایج و تحلیل تجربی

در آزمایش‌های انجام‌شده، ۱۷ مدل مختلف مورد بررسی قرار گرفتند که شامل مدل‌های تجاری مانند GPT-4 و Gemini و مدل‌های متن‌باز مانند DeepSeek و CodeLlama بودند. نتایج نشان دادند که مدل GPT-4o بالاترین عملکرد را دارد و توانسته است به طور میانگین بیش از ۹۸٪ پوشش خطوط و ۹۷٪ پوشش شاخه‌ها را فراهم کند. همچنین مشخص شد که برخی مدل‌های متن‌باز نیز عملکرد نزدیکی به مدل‌های تجاری دارند.

با این حال، در وظایف هدفمند یعنی پوشش خطوط یا شاخه‌های مشخص، فقط مدل‌هایی مانند GPT-4 و GPT-4o توانستند عملکرد بهتری نسبت به وضعیت بدون اطلاع از هدف نشان دهند. سایر مدل‌ها یا هیچ بهبودی نداشتند یا فقط اندکی بهتر عمل کردند. این نشان می‌دهد که مدل‌ها در انجام استدلال منطقی برای رسیدن به نقطه خاصی از کد دچار ضعف هستند.

در وظیفه پوشش مسیر نیز، اکثر مدل‌ها نتوانستند عملکرد چشم‌گیری از خود نشان دهند. حتی با دادن اطلاعات مسیر هدف، خروجی آن‌ها تفاوت زیادی با حالت بدون اطلاعات نداشت. این نشان می‌دهد که فهم دنباله‌های منطقی و تحلیل مسیرهای چندمرحله‌ای در کد، چالشی جدی برای مدل‌های زبانی محسوب می‌شود.

۷. تحلیل تکنیک‌های پیشرفته

برای بهبود عملکرد مدل‌ها، تکنیکی به نام زنجیره تفکر یا Chain-of-Thought (COT) نیز بررسی شد. در این تکنیک، مدل ابتدا باید منطق رسیدن به خط هدف را شرح دهد و سپس تست کیس متناسب با آن بنویسد. این تکنیک توانست در برخی مدل‌ها مانند GPT-4o و LLaMA3 عملکرد را بهبود دهد، اما همچنان در برخی موارد مانند DeepSeek تغییر چشمگیری ایجاد نکرد. در واقع، مدل‌ها در درک منطق درست موفق بودند اما در تبدیل آن به کد عملیاتی دقیق گاهی دچار خطا شدند.

۸. جمع‌بندی و نتیجه‌گیری

مقاله TESTEVAL یک گام مهم در مسیر ارزیابی توانایی مدل‌های زبانی برای تولید تست کیس‌های مؤثر است. این مقاله با طراحی بنچمارکی جامع و دقیق، نه تنها به ارزیابی کلی عملکرد مدل‌ها پرداخته، بلکه آن‌ها را در شرایط خاص و هدفمند نیز بررسی کرده است. نتایج نشان می‌دهد که گرچه مدل‌ها توانایی بالایی در تولید تست‌های متنوع دارند، اما همچنان در تحلیل منطقی و هدفمند کد دچار ضعف هستند. این موضوع اهمیت توسعه روش‌های ترکیبی و هوشمندتر را برای نسل آینده تست کیس‌سازهای خودکار روشن می‌سازد.

۹. محدودیت‌ها

یکی از محدودیت‌های این پژوهش، تمرکز صرف بر زبان پایتون است. در حالی که LLM‌ها در زبان‌های دیگر نیز استفاده می‌شوند، امکان دارد نتایج این بنچمارک برای سایر زبان‌ها تفاوت‌هایی داشته باشد. همچنین استفاده از برنامه‌های LeetCode ممکن است نماینده کامل دنیای واقعی نباشد، اما پیچیدگی منطقی این برنامه‌ها آن‌ها را برای سنجش دقیق توانایی مدل‌ها مناسب کرده است. در نهایت، این پژوهش صرفاً روی برنامه‌های ساده و تک‌فایلی متمرکز است و در صورت گسترش به پروژه‌های واقعی، مسائل امنیتی و وابستگی‌های خارجی باید نیز در نظر گرفته شوند.